

EVA-Ai — Архитектура и Принципы Работы

Репозиторий: <https://github.com/BlackCatSpb/EVA-Ai>

1. Обзор Системы

EVA-Ai — когнитивная нейросетевая система с рекурсивным рассуждением, фрактальной памятью и самообучением. Система построена на архитектуре Three-GGUF Pipeline, где три специализированные модели работают последовательно, обеспечивая качество ответов через перекрёстную проверку и адаптивную настройку параметров.

1.1 Философия Системы

EVA не просто генерирует текст — она **рассуждает**. Каждый ответ проходит через:

- 1. **Логическую проверку** (Model A извлекает факты)
- 2. **Концептуальное развитие** (Model B расширяет мысль)
- 3. **Контроль качества** (проверка на заикание, язык, шаблонность)
- 4. **Семантический анализ** (сравнение с предыдущими попытками через эмбединги)
- 5. **Адаптацию** (корректировка параметров при провалах)

1.2 Ключевые особенности

- **Three-GGUF Pipeline** — три модели (3B + 3B + 1.5B Coder) работают последовательно через llama.cpp, каждая со своей ролью и системным промптом
- **Адаптивная генерация** — AdaptiveParameterController анализирует причины провалов (заикание, фразы-паразиты, китайские символы, семантическая застрялость) и корректирует параметры для следующей попытки
- **Семантический анализ застрялости** — через эмбедек (multilingual-e5-base, 768-dim) система сравнивает семантическую схожесть неудачных ответов; при схожести >0.85 применяется радикальный сдвиг параметров
- **Фрактальная память** — иерархический граф знаний (1046 узлов, 1042 связи) с горячими/холодными узлами, хранящий опыт, концепты и структуры моделей
- **Самообучение** — система автоматически создаёт самодialogи по темам запросов, выявляет пробелы знаний, сохраняет опыт в граф
- **GPU-ускорение** — embedding-модель на CUDA (2GB VRAM), KV-кэш q8_0 для всех моделей (экономия 30% RAM)
- **Структурированные системные промпты** — роль → задача → спецификации → ограничения → формат, с нарастающей строгостью при retry
- **Веб-интерфейс** — Flask сервер с чатом, панелью рассуждений, управлением сессиями, кнопками действий

1.3 Аппаратные требования

Компонент	Требование	Назначение
CPU	6+ ядер (AVX2)	GGUF инференс (llama.cpp)
RAM	16 ГБ минимум	Модели 3B Q4_K_M × 2 = ~4GB, система ~6GB
GPU	CUDA 2+ ГБ VRAM	Embedding-модель (multilingual-e5-base ~1.1GB)
SSD	10+ ГБ свободно	GGUF модели, кэши, граф памяти

2. Архитектура Компонентов

2.1 Ядро (Core) — 8 компонентов

Компонент	Файл	Описание
CoreBrain	core/core_brain.py	Центральный координатор. Загружает brain_config.json, инициализирует Two-Model Pipeline (проверка путей, создание RecursiveModelPipeline), запускает ComponentInitializer, управляет жизненным циклом системы. Обработывает запросы через process_query().
ComponentInitializer	core/component_initializer.py	Фабрика из 23 компонентов. Создаёт компоненты через registered factories, устанавливает зависимости (hybrid_cache → model_manager, ml_unit → text_processor), выполняет пост-инициализацию связей.
RecursiveModelPipeline	core/recursive_model_pipeline.py	Сердце генерации. Загружает Model A/B/C через llama_cpp.Llama(), выполняет последовательную генерацию с quality check, управляет AdaptiveParameterController для каждой модели.
AdaptiveParameterController	core/recursive_model_pipeline.py	Контроллер адаптивных параметров. Хранит историю провалов, вычисляет эмбединги ответов, определяет семантическую застрялость (cosine similarity > 0.85), применяет стратегии адаптации.
EventBus	core/event_bus.py	Pub/sub событийная шина. Компоненты публикуют события (query_processed, response_generated), подписчики реагируют асинхронно.
ResourceManager	core/resource_manager.py	Мониторинг CPU/RAM/GPU в реальном времени. Предупреждения при >90% CPU, >95% RAM.
SystemMonitor	core/system_monitor.py	Health checks компонентов, сбор метрик, алерты.
DeferredCommandSystem	core/deferred_command_system.py	Очередь отложенных команд с 6 воркерами. Выполняет команды после полной инициализации.

2.2 Рассуждение (Reasoning) — 4 компонента

Компонент	Файл	Описание
SelfReasoningEngine	reasoning/self_reasoning_engine.py	Главный движок рассуждения. Получает запрос + историю диалогов, строит контекстный промпт, определяет тип запроса (кратко/подробно/код), запускает Two-Model Pipeline с параметрами.
ReasoningIntegration	reasoning/integration.py	Мост между CoreBrain и SelfReasoningEngine. Передаёт two_model_pipeline напрямую в SRE при создании.
EnhancedReasoningEngine	reasoning/enhanced_reasoning_engine.py	Расширенное рассуждение с подключением ContradictionManager, EthicsFramework, WebSearch.
FractalStorage	reasoning/fractal_ml/fractal_storage.py	Фрактальное хранилище рассуждений (1960 узлов). Сохраняет цепочки рассуждений для последующего анализа.

2.3 Память (Memory) — 4 компонента

Компонент	Файл	Описание
UnifiedFractalMemory	memory/unified_fractal_memory.py	Единая фрактальная память. Граф с 1046 узлами и 1042 связями. Уровни: model_root → component → chunk → layer → tensor. Тип: hot (18) / cold (1028).
GraphLearning	memory/graph_learning.py	Обучение на графе памяти. Сохраняет опыт после каждой генерации, кластеризует опыты, предоставляет контекст для запросов.
HybridTokenCache	memory/hybrid_token_cache.py	Гибридный кэш токенов. RAM 2.8M токенов (11GB), диск 50GB.
EmbeddingCache	memory/embedding_cache.py	Persistent LRU кэш эмбедингов на SQLite (100k записей). Ключ = SHA256 хеш текста. Автоматическая eviction. Ускоряет повторные запросы в 10-100x.

2.4 Машинное обучение (MLearning) — 7 компонентов

Компонент	Файл	Описание
FractalModelManager	mlearning/fractal_model_manager.py	Управление GGUF моделями через llama.cpp. Hot deployment.
HybridModelManager	mlearning/hybrid_model_manager.py	Гибридное управление: VRAM (0.5GB) → SSD (2.0GB).
QwenModelManager	mlearning/qwen_model_manager.py	Управление PyTorch моделями Qwen (qwen3.5-0.8b).
UnifiedTextProcessor	mlearning/unified_text_processor.py	Обработка текста + эмбединги через multilingual-e5-base на GPU. Singleton кэш embedding-моделей.
SentenceTransformersCache	mlearning/sentence_transformers_cache.py	Автодетект CUDA. Функции encode_text() и encode_batch() с автоматическим кешированием.
MLUnit	mlearning/ml_unit.py	ML ядро системы. Координирует ModelManager, TextProcessor, ResponseGenerator.
TokenizerRegistry	mlearning/tokenizer_registry.py	Реестр токенизаторов. Загрузка из локальных путей.

2.5 Генерация, Обучение, Аналитика

Компонент	Файл	Описание
GenerationCoordinator	generation/generation_coordinator.py	Координация генерации. 3 провайдера: HybridModelProvider, ResponseGeneratorProvider, MLUnitProvider.
ResponseGenerator	response_generator.py	Генерация финальных ответов через модель + кэш.
SelfDialogLearning	learning/self_dialog_learning.py	Самодialogи. Создаёт диалоги по темам запросов, сохраняет в граф.
ContradictionManager	contradiction/contradiction_detection.py	Обнаружение противоречий через эмбединги.
EthicsFramework	ethics/ethics_framework.py	7 этических принципов.
KnowledgeGraph	knowledge/knowledge_graph.py	Граф знаний.
WebSearchEngine	web_search.py	Поиск в интернете.

3. Детальный Поток Данных

3.1 Полный цикл обработки запроса

1. WEB GUI (browser → Flask)

Пользователь вводит: "Привет, Ева!"

↓

POST /api/chat {query: "Привет, Ева!", session_id: "abc123"}

↓

WebGUI.process_message()

- └─ EthicsChecker.check_message() → allowed
- └─ EntityExtractor.extract_entities() → []
- └─ add_chat_message(session_id, 'user', "Привет, Ева!")
- └─ add_context_node(session_id, {user_message, entities, ...})
- └─ get_chat_history(session_id, limit=20) → [] (новая сессия)

↓

2. COREBRAIN.process_query(query, user_context)

- └─ Query Analysis:
 - └─ Длина запроса: 12 символов → "подробно"
 - └─ Режим: GGUF (use_two_model_pipeline = true)
- └─ Chat history: [] (0 сообщений)
- └─ Выбор пайплайна: SelfReasoningEngine

↓

3. SELFREASONINGENGINE.process_query()

- └─ _build_contextual_query(query, conversation_history=[])
 - └─ История пуста → возвращает "Привет, Ева!" без изменений
- └─ _determine_query_type() → "подробно"
- └─ Формирование gen_params:
 - model_a: temp=0.2, max_tokens=256, top_p=0.85, top_k=50
 - model_b: temp=0.6, max_tokens=512, top_p=0.85, top_k=50
- └─ pipeline.process_query(query, gen_params)

↓

4. RECURSIVEMODELPIPELINE.process_query()

- └─ Fractal context: get_context_for_query("Привет, Ева!")
 - └─ Пропущено (короткий запрос < 60 символов + приветствие)
- └─ STEP 1: Model A (Логическое Ядро)
 - System prompt: "Ты – Модуль Логического Ядра EVA..."
 - User content: "Привет, Ева!"
 - Params: temp=0.3, top_p=0.9, top_k=40, rep_pen=1.5
 - ↓ create_chat_completion()
 - ↓ _sanitize_response()
 - ↓ check_quality()
 - └─ Проверка: заикливание → OK
 - └─ Проверка: китайские символы → OK
 - └─ Проверка: фразы-паразиты → OK
 - └─ Результат: quality.is_gibberish = False, score = 0.8
 - ↓ record_success()
- └─ STEP 2: Model B (Развитие Концепций)
 - System prompt: "Ты – Модуль Развития Концепций EVA..."
 - User content: "Привет, Ева!"
 - Params: temp=0.3, top_p=0.9, top_k=40, rep_pen=2.0
 - ↓ create_chat_completion()
 - ↓ _sanitize_response() + _clean_filler_start() + _remove_loop
 - ↓ check_quality()
 - └─ Результат: quality.is_gibberish = False
- └─ STEP 3: Model C (Кодек) – пропущен (нет code-ключевых слов)

5. POST-PROCESSING

```
|— add_chat_message(session_id, 'assistant', response_text)
|— add_context_node(session_id, {assistant_message, reasoning})
|— GraphLearning: сохранение опыта (quality=0.80)
|— SelfDialogLearning: создание самодialogа по теме "Привет, Ева!"
|— Return response → Web GUI → отображение пользователю
```

3.2 Адаптивная генерация — детальный сценарий провала

Запрос: "Проанализируй стихотворение..."

```
Model A – Attempt 1
Params: temp=0.30, rep=1.50, top_k=40, top_p=0.90
Result: "Конечно! Давайте разберём..."
Quality check: 'Начинается с фразы-паразита' → FAIL
record_failure(reasons=['фразы-паразиты'], text="Конечно!...")
Embedding computed → saved to failed_response_embeddings[]
```

```
Model A – Attempt 2
get_params_for_attempt(attempt=2, failure_reasons=['фразы...'])
Rule-based: 'фраз' in reason → ↑temp +0.2, ↑top_k +20
Adapted params: temp=0.50, rep=1.70, top_k=60, top_p=0.90
System prompt MODIFIED:
"ЗАПРЕЩЕНО начинать с: «Конечно», «Давайте», «Вот»..."
Result: "Конечно, разберём..."
Quality check: 'Начинается с фразы-паразита' → FAIL
record_failure(...)
Embedding computed → saved
Semantic stuck check: similarity(emb1, emb2) = 0.94 > 0.85 → STUCK
stuck_factor = 0.50
```

```
Model A – Attempt 3
get_params_for_attempt(attempt=3, failure_reasons=['фразы...'])
Rule-based: ↑temp +0.2
Semantic: stuck_factor=0.50 → радикальный сдвиг:
temp = 0.30 + 0.2 + 0.3 + 0.5*0.3 = 0.95
rep_pen = 1.50 + 0.2 + 0.3 + 0.5*0.3 = 2.15
top_k = 40 + 20 + 20*0.5 = 70
Adapted params: temp=0.95, rep=2.15, top_k=70, top_p=0.90
System prompt: ещё строже
Result: "Стихотворение содержит..." ✓ SUCCESS
record_success()
```

3.3 Стратегии адаптации (полная таблица)

Причина провала	temperature	top_p	top_k	repeat_penalty	max_tokens	Системный промпт
Зацикливание	+0.25	-0.1	-10	+0.4	—	"Каждое предложение должно нести новую информацию"
Китайские символы	-0.15	-0.2	—	+0.3	—	"ЗАПРЕЩЕНО: использовать китайские символы"
Фразы-паразиты	+0.2	—	+20	+0.2	—	"ЗАПРЕЩЕНО начинать с: «Конечно», «Давайте»"

Причина провала	temperature	top_p	top_k	repeat_penalty	max_tokens	Системный промпт
Пустой ответ	+0.15	—	—	—	+256	—
Нет гласных	+0.35	—	—	+0.3	—	—
Семантическая застрялость	+0.3 + stuck*0.3	+0.1*(1-stuck)	+20*stuck	+0.3 + stuck*0.3	—	Максимальная строгость

4. Память и Обучение

4.1 UnifiedFractalMemory — полная структура

Уровень 0: model_root (6 узлов)
├─ model_a — узел модели A
├─ model_b — узел модели B
├─ model_c — узел модели C
└─ ...

Уровень 1: component (12 узлов)
├─ query_processor
├─ response_generator
├─ reasoning_engine
├─ memory_manager
└─ ...

Уровень 2: chunk (28 узлов)
├─ Фрагменты текстов
├─ Концепты
└─ ...

Уровень 3: layer (100 узлов)
├─ Слои абстракции
└─ ...

Уровень 4: tensor (900 узлов)
├─ Веса и параметры
└─ ...

Тир доступа:

- **Hot** (18 узлов) — загружены в RAM, быстрый доступ
- **Cold** (1028 узлов) — на SSD, загружаются по запросу

Операции:

- `get_context_for_query(query)` — поиск релевантных узлов
- `register_model_instance(name, model)` — регистрация модели в графе
- `add_experience(query, response, quality)` — сохранение опыта

4.2 Graph Learning — цикл обучения

После каждой генерации:

1. Сохранение опыта

- ├─ Model A ответ → опыт (quality=0.80)
- └─ Model B ответ → опыт (quality=0.00)

2. Кластеризация

- ├─ Группировка опытов по темам
- └─ Выявление паттернов

3. Обновление графа

- └─ Создание новых узлов

- └ Обновление связей
- 4. Выявление пробелов знаний
 - └ LearningOpportunityManager

4.3 Self-Dialog Learning

После каждого запроса пользователя:

1. Создание темы самодialogа
topic = query[:100]
context = {user_query, system_response}
2. Внутренний диалог
EVA задаёт вопросы себе по теме
Анализирует ответы
3. Сохранение результатов
 - └ В граф памяти
 - └ Выявление пробелов знаний
4. Learning Opportunities
 - └ Создание задач для дообучения

4.4 Embedding Cache — архитектура

SQLite Database: eva/memory/embedding_cache/embeddings.db

Table: embeddings

└ hash TEXT PRIMARY KEY	└ SHA256 хеш текста
└ embedding TEXT NOT NULL	└ JSON array (768 floats)
└ text_preview TEXT	└ первые 100 символов
└ dimension INTEGER	└ 768
└ created_at TEXT	└ ISO timestamp
└ accessed_at TEXT	└ последний доступ
└ access_count INTEGER	└ счётчик обращений

Index: idx_accessed_at — для LRU eviction

Max size: 100,000 записей

Eviction: удаляются самые старые по accessed_at

API:

```
from eva.memory.embedding_cache import get_embedding_cache
cache = get_embedding_cache()

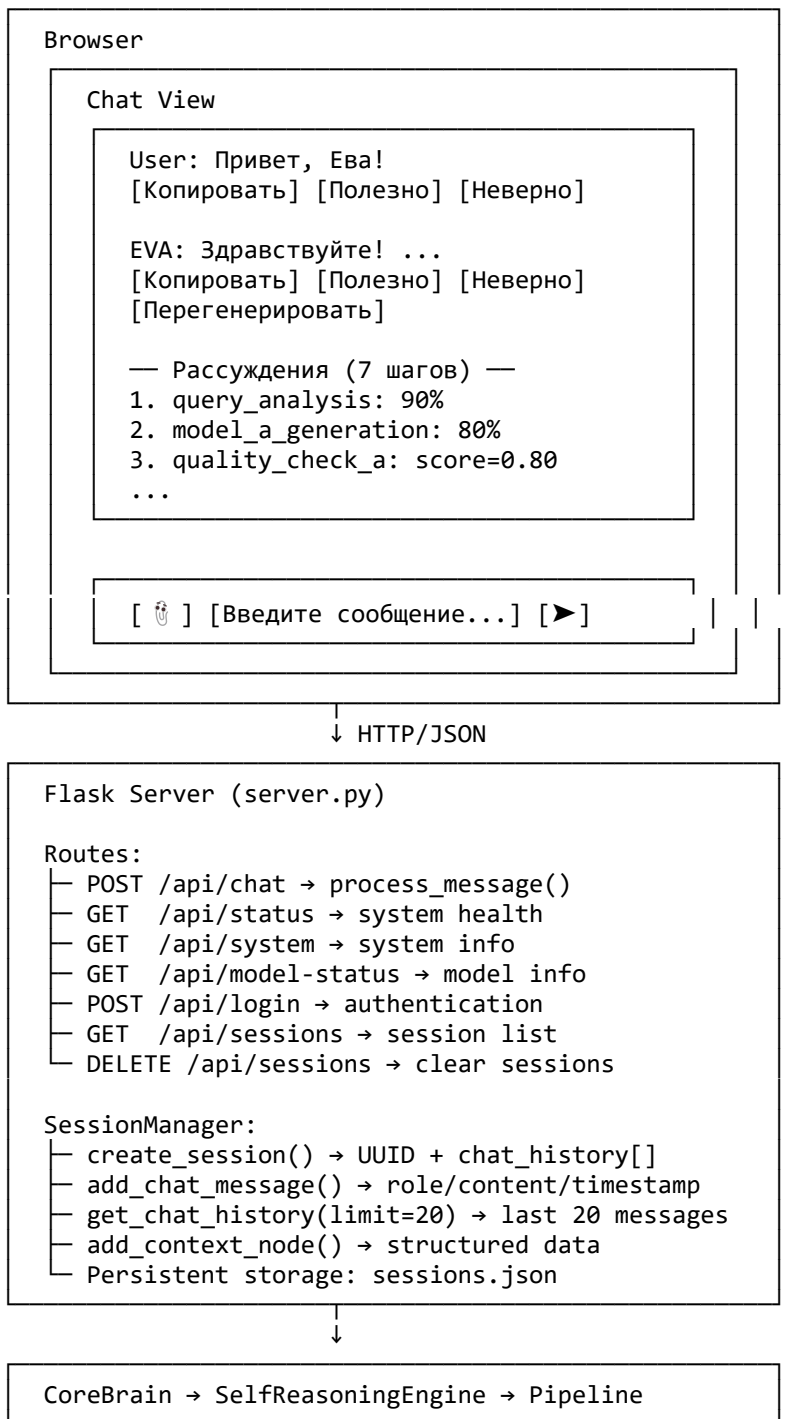
# Получить или вычислить
embedding = cache.get_or_compute("Привет!", lambda t: model.encode([t])[0])

# Batch
results = cache.batch_get(["текст1", "текст2"])

# Статистика
stats = cache.get_stats() # {count, size_mb, dimension, hit_rate}
```

5. Веб-интерфейс

5.1 Архитектура



5.2 Сессии и История — детально

chat_history[] — сырая история чата:

```
[
  {"role": "user", "content": "Привет, Ева!", "timestamp": "2026-04-03T17:15:23"},
  {"role": "assistant", "content": "Здравствуйте! ...", "timestamp": "2026-04-03T17:16:24"}
]
```

- Используется **только** для контекста генерации
- Передаётся в `SelfReasoningEngine._build_contextual_query()`
- Последние 10 сообщений включаются в промпт
- Максимум 50 сообщений хранится

`context_nodes[]` — структурированные узлы:

```
[
  {
    "id": "uuid-1",
    "user_message": "Привет, Ева!",
    "timestamp": "2026-04-03T17:15:23",
    "entities": [],
    "file_data": null
  },
  {
    "id": "uuid-2",
    "assistant_message": "Здравствуйте! ...",
    "timestamp": "2026-04-03T17:16:24",
    "reasoning": "Query analysis: подробно..."
  }
]
```

- Используются для **анализа, обучения, самодialogа**
- **Не попадают** в промпт генерации

5.3 Кнопки действий

Кнопка	Действие
Копировать	<code>navigator.clipboard.writeText(text)</code>
Полезно	<code>rateMessage(text, 1)</code> — положительная оценка
Неверно	<code>rateMessage(text, -1)</code> — отрицательная оценка
Перегенерировать	Находит предыдущий user-запрос, удаляет текущий ответ, вызывает <code>sendMessage(userText)</code> заново

6. Конфигурация

6.1 `brain_config.json` — полная структура

```
{
  "model": {
    "use_two_model_pipeline": true,
    "use_llama_cpp": false,
    "model_a_gguf_path": "eva/memory/fractal_torch_storage/gguf_models/qwen2.5-3b-instruct/qwen2.5-3b-instruct-gguf",
    "model_b_gguf_path": "eva/memory/fractal_torch_storage/gguf_models/qwen2.5-3b-instruct/qwen2.5-3b-instruct-gguf",
    "model_c_gguf_path": "eva/memory/fractal_torch_storage/gguf_models/qwen2.5-coder-1.5b-instruct/qwen2.5-coder-1.5b-instruct-gguf",
    "n_ctx": 8192,
    "n_threads": 8
  },
  "reasoning": {
    "enabled": true,
    "max_iterations": 5,
    "confidence_threshold": 0.75,
    "store_reasoning_chains": true
  },
  "embedding": {
    "model_name": "intfloat/multilingual-e5-base",
    "device": "auto"
  },
  "web_gui": {
    "host": "127.0.0.1",
    "port": 5555,
    "admin_password": "..."
  }
}
```

```
}  
}
```

6.2 Системные промпты — детально

Model A (Логическое Ядро) — базовый:

Ты — Модуль Логического Ядра EVA.

Задача: Извлекать точные факты из запроса без расширений.

Спецификации:

1. Отвечай только подтверждёнными фактами.
2. Максимум 3 предложения.
3. Не используй слова «возможно», «вероятно», «может быть».
4. Избегай оценочных суждений.
5. Если информации недостаточно — сообщи об этом прямо.

Ограничения:

- Не добавляй вступления или заключения.
- Не повторяй вопрос пользователя.
- Отвечай строго на русском языке.

Формат вывода: Русский. Ответ начинается сразу с факта.

Конец инструкции.

Model A — при фразе-паразите (retry):

Ты — Модуль Логического Ядра EVA.

Задача: Отвечай строго по существу, без вступлений.

ЗАПРЕЩЕНО начинать с фраз: «Конечно», «Давайте», «Вот», «Это», «Привет», «Здравствуйте».

Спецификации:

1. Начни ответ сразу с факта или прямого ответа.
2. Максимум 3 предложения.
3. Не используй вводные слова и фразы-паразиты.
4. Отвечай строго на русском языке.
5. Избегай общих фраз — только конкретика.

Ограничения:

- Никаких вступлений, приветствий, обращений.
- Не повторяй вопрос пользователя.

Формат вывода: Русский. Сразу с факта.

Конец инструкции.

Model A — при заикливании (retry):

Ты — Модуль Логического Ядра EVA.

Задача: Дай уникальный ответ без повторений.

Спецификации:

1. Каждое предложение должно нести новую информацию.
2. Запрещено повторять одинаковые мысли.
3. Максимум 3 предложения.
4. Отвечай строго на русском языке.
5. Используй разнообразные конструкции.

Ограничения:

- Не повторяй предложения или их части.
- Не используй одинаковые начала предложений.

Формат вывода: Русский. Сразу с факта.

Конец инструкции.

Model B (Развитие Концепций) — базовый:

Ты — Модуль Развития Концепций EVA.

Задача: Развивай мысль, добавляй детали и примеры.

Спецификации:

1. Расширяй факты примерами и пояснениями.

2. Используй структурированный формат (списки, абзацы).
3. Отвечай строго на русском языке.
4. Добавляй контекст и смежные темы.
5. Максимум 10 предложений.

Ограничения:

- Не повторяй факты дословно.
- Не используй английские или китайские вставки.

Формат вывода: Русский, развёрнутый ответ.

Конец инструкции.

7. Защита и Безопасность

7.1 Singleton Protection

```
# eva/run.py
_PID_FILE = ".eva_instance.pid"

def _check_singleton():
    if os.path.exists(_PID_FILE):
        old_pid = int(open(_PID_FILE).read())
        # Проверяем, жив ли процесс через tasklist
        if process_alive(old_pid):
            sys.exit(1) # Уже запущено
        os.remove(_PID_FILE) # Stale PID

    open(_PID_FILE, "w").write(str(os.getpid()))
    atexit.register(_cleanup_pid) # Удаление при завершении
```

7.2 Этическая рамка

7 принципов проверяют каждый запрос и ответ:

1. Не причиняй вред
2. Уважай автономию
3. Будь честной
4. Защищай конфиденциальность
5. Будь справедливой
6. Неси ответственность
7. Стремись к благу

7.3 Обнаружение противоречий

- Эмбеддер сравнивает новый ответ с сохранёнными знаниями
 - При cosine similarity < порога — flagged как потенциальное противоречие
 - Сохраняется в contradictions.db для анализа
-

8. Запуск

```
python -m eva
```

Последовательность запуска:

1. eva/__main__.py — настройка логирования, вызов run.main()
2. eva/run.py — _check_singleton(), инициализация CoreBrain
3. CoreBrain.__init__() — загрузка config, инициализация Two-Model Pipeline
4. CoreBrain.initialize() — ComponentInitializer (23 компонента)

5. `CoreBrain.start()` — запуск компонентов, EventBus, фоновых задач
6. `launch_gui()` — Flask сервер на <http://127.0.0.1:5555>

Что запускается:

- CoreBrain с 26 компонентами (10 активных, 16 пассивных)
- Two-Model Pipeline: Model A (3B) + Model B (3B) + Model C lazy
- Embedding модель: multilingual-e5-base на GPU (если CUDA доступна)
- Web GUI: Flask + HTML/CSS/JS
- Фоновые задачи: self-dialog (каждую минуту), graph curator, monitoring (каждые 5 сек)

9. Структура Директорий

CogniFlex/	
eva/	
core/	# Ядро системы
core_brain.py	# Главный координатор (3620 строк)
recursive_model_pipeline.py	# Three-GGUF pipeline + AdaptiveController
component_initializer.py	# Фабрика 23 компонентов
event_bus.py	# Pub/sub событийная шина
resource_manager.py	# Мониторинг ресурсов
system_monitor.py	# Health checks
deferred_command_system.py	# Отложенные команды
generation_coordinator.py	# Координация генерации
integration_layer.py	# Слой интеграции
memory_graph_ml.py	# ML на графе памяти
eva_cache/	# Кэш ядра
token_cache/	# TokenDiskCache
hybrid_cache/	# HybridTokenCache
reasoning/	# Рассуждение
self_reasoning_engine.py	# SelfReasoningEngine (1780 строк)
integration.py	# ReasoningIntegration
enhanced_reasoning_engine.py	# EnhancedReasoningEngine
analytics_module.py	# Аналитика рассуждений
prompt_composer.py	# Композитор промптов
semantic_stability.py	# Проверка стабильности
combined_metric.py	# Комбинированные метрики
entity_extractor.py	# Извлечение сущностей
correlation.py	# Корреляционный анализ
fractal_ml/	# Фрактальное хранилище
fractal_storage.py	# 1960 узлов
fractal_embedder.py	# Эмбединги
fractal_retriever.py	# Извлечение
fractal_tokenizer.py	# Токенизация
memory/	# Память
unified_fractal_memory.py	# Единая фрактальная память
graph_learning.py	# Обучение на графе
hybrid_token_cache.py	# Гибридный кэш токенов
embedding_cache.py	# SQLite LRU кэш эмбедингов
mllearning/	# ML компоненты
fractal_model_manager.py	# Управление GGUF моделями
hybrid_model_manager.py	# Гибридное управление
qwen_model_manager.py	# Управление PyTorch Qwen
unified_text_processor.py	# Текст + эмбединги
sentence_transformers_cache.py	# Кэш embedding-моделей
ml_unit.py	# ML ядро
tokenizer_registry.py	# Реестр токенизаторов
model_selector.py	# Селектор моделей
eva_models/	# PyTorch модели
qwen3.5-0.8b/	# Qwen 0.8B
gui/web_gui/	# Веб-интерфейс

server.py	# Flask сервер (1724 строки)
templates/	
└ index.html	# HTML шаблон
static/	
└ css/style.css	# Стили
└ js/app.js	# JavaScript логика
uploads/	# Загруженные файлы
learning/	# Обучение
└ self_dialog_learning.py	# Самодialogи
└ learning_manager.py	# Управление обучением
└ integrated_learning_manager.py	# Интегрированное обучение
knowledge/	# Знания
└ knowledge_graph.py	# Граф знаний
└ knowledge_storage.py	# Хранилище знаний
└ knowledge_search.py	# Поиск знаний
└ qwen_api_enhancer.py	# Обогащение через API
contradiction/	# Противоречия
└ contradiction_detection.py	# Обнаружение
└ contradiction_learning.py	# Обучение на противоречиях
ethics/	# Этика
└ ethics_framework.py	# 7 принципов
analytics/	# Аналитика
└ analytics_manager.py	# Менеджер аналитики
└ contradiction_analyzer.py	# Анализ противоречий
generation/	# Генерация
└ generation_coordinator.py	# Координатор
tools/	# Инструменты
└ import_pipeline.py	# Импорт данных
└ layer_expertise_analysis.py	# Анализ слоёв модели
adaptation/	# Адаптация
└ core.py	# Менеджер адаптации
monitoring/	# Мониторинг
└ system_monitor.py	# Системный монитор
preprocess/	# Препроцессинг
└ pipeline.py	# Пайплайн обработки
response_generator.py	# Генератор ответов
analyzer_core.py	# Ядро анализа
performance_analyzer.py	# Анализ производительности
web_search.py	# Поиск в интернете
graph_curator.py	# Куратор графа
gguf_training.py	# Обучение GGUF
self_analyzer.py	# Самоанализ
__main__.py	# Точка входа
brain_config.json	# Конфигурация системы
DESIGN.md	# Архитектура и принципы
README.md	# Описание проекта
requirements.txt	# Зависимости
pyproject.toml	# Метаданные проекта
logs/	# Логи
└ eva.log	# Основной лог
eva/memory/	# Фрактальная память
└ fractal_torch_storage/	# Хранилище моделей
└ └ gguf_models/	# GGUF модели
└ └ └ qwen2.5-3b-instruct/	# Model A/B (3B)
└ └ └ qwen2.5-coder-1.5b-instruct/	# Model C (1.5B)
└ └ └ qwen2.5-0.5b-instruct-q4_0.gguf	# LlamaCpp
└ unified_memory/	# UnifiedFractalMemory

└─ nodes.json	# Узлы графа
└─ edges.json	# Связи графа
─ eva/knowledge/	# База знаний
└─ eva_knowledge_cache/	
└─ knowledge_graph.db	# SQLite граф знаний
─ eva/core/eva_cache/	# Кэш ядра
└─ token_cache/	# TokenDiskCache
└─ hybrid_cache/	# HybridTokenCache
└─ ml_unit/hybrid_cache/	# MLUnit кэш
└─ fractal_reasoning/	# FractalStorage
─ eva/cache/	# Кэш противоречий
└─ contradictions.db	# SQLite противоречия
─ .eva_instance.pid	# PID-файл (singleton)
─ .gitignore	# Git ignore
─ start_webgui.py	# Альтернативный запуск GUI

10. Качество Генерации

10.1 Quality Check — детально

```
def check_quality(text: str) -> Dict[str, Any]:
    """Проверка качества текста с детекцией заикливания"""

    # 1. Пустой текст
    if len(text.strip()) < 5:
        return {'is_gibberish': True, 'score': 0.1, 'reasons': ['Пустой']}

    # 2. Повторения слов
    words = text.split()
    unique = set(words)
    if len(words) > 10 and len(unique) / len(words) < 0.3:
        return {'is_gibberish': True, 'score': 0.2, 'reasons': ['Повторения слов']}

    # 3. Нет гласных
    if not any(v in text for v in 'aeёиоуыэюяАЕЁИОУЫЭЮЯaeiouAEIOU'):
        return {'is_gibberish': True, 'score': 0.1, 'reasons': ['Нет гласных']}

    # 4. Заикливание предложений
    lines = text.split('\n')
    repeating = {}
    for line in lines:
        if len(line.strip()) > 20:
            repeating[line.strip()] = repeating.get(line.strip(), 0) + 1
    if max(repeating.values(), default=1) > 2:
        return {'is_gibberish': True, 'score': 0.3, 'reasons': ['Заикливание']}

    # 5. Фразы-паразиты
    for filler in ['Конечно!', 'Конечно', 'Вот более', 'Вот что', 'Это всё']:
        if text.startswith(filler):
            return {'is_gibberish': True, 'score': 0.4, 'reasons': ['Фраза-паразит']}

    # 6. Китайские символы
    if sum(1 for c in text if '一' <= c <= '隹') > 5:
        return {'is_gibberish': True, 'score': 0.1, 'reasons': ['Китайские символы']}

    # 7. Много английских слов
    english = sum(1 for w in words if w.isascii() and len(w) > 3)
    if len(words) > 10 and english / len(words) > 0.3:
        return {'is_gibberish': True, 'score': 0.3, 'reasons': ['Много английских']}
```

```
# OK
return {'is_gibberish': False, 'score': 0.8, 'reasons': ['OK']}
```

10.2 Семантический анализ застрялости

```
def _are_embeddings_stuck(self) -> Dict[str, Any]:
    """Проверяет, застряли ли мы в семантически одинаковых ответах"""
    if len(self.failed_response_embeddings) < 2:
        return {'is_stuck': False, 'max_similarity': 0.0, 'stuck_count': 0}

    # Сравниваем последний эмбединг со всеми предыдущими
    max_sim = 0.0
    for i in range(len(self.failed_response_embeddings) - 1):
        sim = cosine_similarity(embeddings[i], embeddings[-1])
        max_sim = max(max_sim, sim)

    stuck_count = sum(1 for emb in embeddings[:-1]
                      if cosine_similarity(emb, embeddings[-1]) > 0.85)

    return {
        'is_stuck': max_sim > 0.85,
        'max_similarity': max_sim,
        'stuck_count': stuck_count,
    }
```
